

אינטגרציית SmartConsole מול SolarWinds – מוסבר

איך סקריפטי הגילוי של Check Point SmartConsole וסקריפטי יצירת ה-Node ב-SolarWinds שברופו הזה מתחברים יחד.

1. מה האוטומציה הזו עושה

המטרה היא לקחת את כתובות ה-IP של חברי הפיירוול ששייכים לאובייקט ClusterXL (מסוג `simple-cluster`) בשרת הניהול של Check Point, ולרשום כל חבר ככה כ-Node מנוטר ב-SolarWinds. כדי לעשות את זה בצורה אמينة צריך לדבר עם שתי מערכות שאין ביניהן קשר, לכל אחת שיטת אימות וסגנון API משלה – ולכן העבודה מפוצלת לארבעה סקריפטים ולא לאחד גדול:

סקריפט	תפקיד
<code>CheckPoint.GetClusterFirewallAddresses.ps1</code>	מתחבר ל-Management API של Check Point (ה-API שמאחורי SmartConsole) ומחזיר את כתובות ה-IP של חברי הקלאסטר כאובייקטי PowerShell.
<code>CRUD.AddNodeWithPollers.ps1</code>	פונקציית עזר משותפת: מוסיפה Node אחד ב-SolarWinds ורושמת לו את הפולרים (pollers) המובנים בברירת המחדל. לא מכילה שום לוגיקה של Check Point.
<code>CRUD.AddNodesFromCsv.ps1</code>	צרכן מס' 1 – קורא קובץ CSV (זוגות שם/IP) ומוסיף כל שורה כ-Node ב-SolarWinds.
<code>Orchestrator.AddClusterFirewallsToSolarWinds.ps1</code>	צרכן מס' 2 – קורא ישירות לפונקציית הגילוי של Check Point ומזרים את התוצאות ישר ל-SolarWinds, בלי קובץ ביניים.

למה ארבעה קבצים ולא אחד

צד ה-Check Point וצד ה-SolarWinds נשארים שני חלקים נפרדים, שכל אחד מהם ניתן לשימוש חוזר בפני עצמו (פונקציות שעושים להן dot-source), במקום סקריפט מונוליטי אחד. זה קונה שלושה דברים:

- **לוחות זמנים וביקורת עצמאיים.** הגילוי יכול לרוץ בקצב משלו, ואת הרשימה שמתקבלת אפשר לבדוק לפני שמוסיפים משהו לניטור.

- **בדיקה עצמאית.** כל צד אפשר להריץ (או לדמות) בלי לצטרכ את פרטי ההתחברות של הצד השני.

- **שימוש חוזר.** ל-`Add-SwisNodeWithDefaultPollers` לא אכפת מהיכן הגיעה כתובת ה-IP – זו בדיוק אותה פונקציה שמשתת גם את `CRUD.AddNodesFromCsv.ps1` וגם את האורקסטרטור.

שני סקריפטי הצריכה מייצגים את שני הקצוות של הפשרה הזו: מבוסס-CSV לשער ביקורת ידני, מבוסס-אורקסטרטור לריצה אוטומטית אחת (למשל משימה מתוזמנת).

`CheckPoint.GetClusterFirewallAddresses.ps1`

מדבר עם ה-API של Management של Check Point — ה-REST API (/web_api/) שגם SmartConsole עצמו הוא לקוח שלו. הקובץ חושף שלוש פונקציות ועוד אחת שמרכיבה אותן יחד.

Connect-SmartConsole

מתחבר עם `POST /web_api/v1.9/login`, ושולח את שם המשתמש ועותק מפוענח-בזיכרון של הסיסמה שהגיעה כ-`SecureString`. השרת מחזיר מזהה session (`sid`), שמוחזר בתוך אובייקט session קטן (`Server`), שכל קריאה מאוחרת יותר צריכה. (Port, Sid, TrustAllCertificates)

```
$body = @{ user = $Username; password = $plainPassword } | ConvertTo-Json
$uri = "https://{Server}:{Port}/web_api/v1.9/login"
$response = Invoke-RestMethod -Uri $uri -Method Post -Body $body -ContentType "application/json"
# $response.sid is the session id used for every subsequent call
```

Invoke-SmartConsoleApi

עטיפה דקה סביב `Invoke-RestMethod` שדרכה עוברת כל קריאת API אחרת: היא בונה את ה-URL משם הפקודה, מקודדת את הגוף כ-JSON, ומצרפת את מזהה ה-session ככותרת `X-chkp-sid` שה-API דורש.

Get-SmartConsoleClusterFirewallAddresses

לוגיקת הגילוי בפועל. אובייקטי ClusterXL של Check Point מיוצגים על-ידי סוג האובייקט `simple-cluster`. הפונקציה הזו:

- קוראת ל-`show-simple-clusters` כדי לעבור על כל אובייקטי הקלאסטר בדפדוף (50 בכל פעם, לפי מוסכמת הדפדוף `total / offset / limit` של ה-API).
- עבור כל קלאסטר, קוראת ל-`show-simple-cluster` עם `details-level=full` כדי לקבל את רשימת החברים המלאה.
- משטחת את התוצאה: שורת פלט אחת לכל חבר קלאסטר, שנושאת את שם הקלאסטר ואת ה-VIP שלו לצד השם וכתובת ה-IP של אותו חבר עצמו.

```
foreach ($member in $cluster.'cluster-members') {
    $results += [pscustomobject]@{
        ClusterName = $cluster.name
        ClusterVip   = $cluster.'ipv4-address'
        MemberName   = $member.name
        MemberIp     = $member.'ip-address'
    }
}
```

הם בדיוק שמות העמודות ששני סקריפטי הצריכה ב-SolarWinds מצפים להם — השיתוף הזה בשמות הוא מה שמאפשר גם למסירה דרך CSV וגם למסירה הישירה דרך האורקסטרטור לעבוד בלי שלב תרגום.

Disconnect-SmartConsole

קוראת ל-`logout` כדי לבטל את מזהה ה-session. חשוב לקרוא לזה תמיד בתוך בלוק `finally`, כדי שגילוי שנכשל לא ישאיר session פתוח דולף.

תעודות SSL: כברירת מחדל הסקריפט **לא** עוקף אימות TLS. שרתי ניהול של Check Point בדרך כלל מציגים תעודה חתומה עצמית, ולכן יש switch בשם `-TrustAllCertificates` — זמין — אבל זו בחירה מפורשת (opt-in) ולא ברירת מחדל, כי לסמוך על כל תעודה בשקט מבטל את כל התכלית של TLS.

CRUD.AddNodeWithPollers.ps1

פונקציה אחת, `Add-SwisNodeWithDefaultPollers`, שהוצאה מתוך דוגמת ה-`CRUD.AddNode.ps1`, המקורית כדי ששני סקריפטי הצריכה למטה יוכלו לקרוא לה במקום כל אחד לשכפל בעצמו את אותן קריאות SWIS.

```
function Add-SwisNodeWithDefaultPollers {
    param($Swis, [string]$IPAddress, [string]$Name = "",
        [string]$Community = "public", [string]$RWCommunity = "")

    $newNodeProps = @{
        IPAddress = $IPAddress; EngineID = 1; Caption = $Name;
        ObjectSubType = "SNMP"; SNMPVersion = 2;
        Community = $Community; RWCommunity = $RWCommunity;
        DNS = ""; SysName = "";
    }
    $newNodeUri = New-SwisObject $Swis -EntityType "Orion.Nodes" -Properties $newNodeProps
    $nodeProps = Get-SwisObject $Swis -Uri $newNodeUri

    foreach ($pollerType in @("N.Status.ICMP.Native", "N.ResponseTime.ICMP.Native",
        "N.Details.SNMP.Generic", "N.Uptime.SNMP.Generic")) {
        New-SwisObject $Swis -EntityType "Orion.Pollers" -Properties @{
            NetObject = "N:" + $nodeProps["NodeID"]; NetObjectType = "N"
            NetObjectID = $nodeProps["NodeID"]; PollerType = $pollerType
        } | Out-Null
    }
    $nodeProps
}
```

`Community` הוא מחרוזת ה-`community` לקריאה (SNMP v2) ש-SolarWinds משתמש בה כדי לתשאל את המכשיר; ברירת המחדל היא `"public"`, אבל זה פרמטר כדי שפיירוולים אמיתיים (שלעולם לא צריך להשאיר על ה-`community` המחדל) יוכלו להעביר משלהם. `RWCommunity` היא ה-`community` לכתובה, אופציונלית, ונשארת ריקה אלא אם SolarWinds צריך גם להגדיר את המכשיר, לא רק לנטר אותו.

נדרשות שתי קריאות SWIS לכל Node (`New-SwisObject` ואז `Get-SwisObject`) כי קריאת היצירה מחזירה רק URI — ה-`NodeID` שמשמש לרישום הפולרים צריך להישלף בחזרה אחר-כך.

CRUD.AddNodesFromCsv.ps1

הצרכן ה"מנותק". קורא קובץ CSV (כברירת מחדל `.\ClusterFirewalls.csv`) — הקובץ שה-`Export-Csv` בדוגמת השימוש של סקריפט ה-`SmartConsole` מייצר) וקורא לפונקציה המשותפת פעם אחת עבור כל שורה.

```
$rows = Import-Csv -Path $CsvPath
foreach ($row in $rows) {
    $name = $row.$NameColumn # default column: MemberName
    $ip = $row.$IpColumn # default column: MemberIp
    if ([string]::IsNullOrEmpty($ip)) { continue }
    Add-SwisNodeWithDefaultPollers -Swis $swis -Name $name -IPAddress $ip -Community $Community
}
```

שמות העמודות (-IpColumn / -NameColumn) הם פרמטרים ולא קבועים בקוד, כך שאותו סקריפט עובד מול CSV מכל מקור, לא רק מזה של Check Point.

מכיוון שה-CSV יושב בין שתי המערכות כקובץ פשוט, הוא משמש גם כתיעוד ביקורת (audit trail): אפשר לפתוח אותו, למחוק שורה שלא רוצים לנטר, או להשוות אותו לקובץ של אתמול לפני שמריצים את הסקריפט הזה.

Orchestrator.AddClusterFirewallsToSolarWinds.ps1

הצרכן ל"ריצה אחת". עושה dot-source לשני קבצי הפונקציות, ואז מחבר אותם יחד בזיכרון בלי שום קובץ CSV בכלל:

```
. (Join-Path $PSScriptRoot "CheckPoint.GetClusterFirewallAddresses.ps1")
. (Join-Path $PSScriptRoot "CRUD.AddNodeWithPollers.ps1")

$cpSession = Connect-SmartConsole -Server $cpServer -Username $cpUsername -Password $cpPassword
try { $firewalls = Get-SmartConsoleClusterFirewallAddresses -Session $cpSession }
finally { Disconnect-SmartConsole -Session $cpSession }

$swis = Connect-Swis -host $swHostname -cred $swCred
foreach ($fw in $firewalls) {
    Add-SwisNodeWithDefaultPollers -Swis $swis -Name $fw.MemberName `
        -IPAddress $fw.MemberIp -Community $swCommunity
}
```

Dot-sourcing (הנקודה בתחילת השורה) מריץ את הקובץ היעד ב-scope הנוכחי במקום ב-scope ילד, כך שהפונקציות שהוא מגדיר (Connect-SmartConsole, Get-SmartConsoleClusterFirewallAddresses, Add-SwisNodeWithDefaultPollers וכו') הופכות לזמינות לקריאה כאן. אף אחד מהקבצים שעושים להם source לא מריץ את בלוק דוגמת השימוש שלו, כי הבלוק הזה מוער (comment) בתחתית כל קובץ — dot-sourcing רק מגדיר את הפונקציות.

הסקריפט הזה צריך גם את פרטי ההתחברות ל-Check Point וגם את אלה של SolarWinds זמינים באותה ריצה, והוא מוסיף Node-ים ברגע שהם מתגלים — אין שלב ביקורת. כדאי להשתמש במסלול ה-CSV כשהפעם הזה חשוב (למשל ניטור בפרודקשן, שבו טעות תצורה בקלאסטר שלא נבדקה עלולה אחרת להיכנס לניטור אוטומטית).

6. הזרימה מקצה לקצה, שני המסלולים

שלב	מסלול עם ביקורת (CSV)	מסלול אוטומטי (אורקסטרטור)
1	הרצת <code>Export-Csv</code> , <code>CheckPoint.GetClusterFirewallAddresses.ps1</code> לתוצאה	הרצת סקריפט האורקסטרטור

שלב	מסלול עם ביקורת (CSV)	מסלול אוטומטי (אורקסטרטור)
2	פתיחה/בדיקה של <code>ClusterFirewalls.csv</code>	(הגילוי קורה בזיכרון, אין שלב ביקורת)
3	הרצת <code>CRUD.AddNodesFromCsv.ps1 -CsvPath ClusterFirewalls.csv</code>	ה-Node ים נוספים מיד
4	Node ים ופולרים נוצרים ב-SolarWinds	Node ים ופולרים נוצרים ב-SolarWinds

דברים לשנות לפני הרצה אמיתית

- למלא שמות שרתים/פרטי התחברות אמיתיים ל-Check Point ול-SolarWinds — כל סקריפט מגיע עם ערכי placeholder ומצפה שתערכו אותם (בהתאם למוסכמה של כל סקריפט אחר בריפו הזה, למשל `CRUD.AddNode.ps1`).
- להגדיר מחרוזת SNMP community אמיתית לכל סביבה — `"public"` היא ברירת מחדל, לא המלצה.
- להחליט במפורש על `-TrustAllCertificates` אם שרת הניהול של Check Point משתמש בתעודה חתומה עצמית, במקום להעביר אותו תמיד.

נוצר כדי לתעד את הסקריפטים תחת `Scripts/` בריפו הזה: `CRUD.AddNodeWithPollers.ps1`, `CheckPoint.GetClusterFirewallAddresses.ps1`, `CRUD.AddNodesFromCsv.ps1` ו-`Orchestrator.AddClusterFirewallsToSolarWinds.ps1`.